MEMORIA: OPTIMIZACIÓN, DOCUMENTACIÓN Y CONTROL DE VERSIONES

Álvaro Vázquez y Jesus Cinta

1. Configuración del Repositorio GIT (Tarea 1)

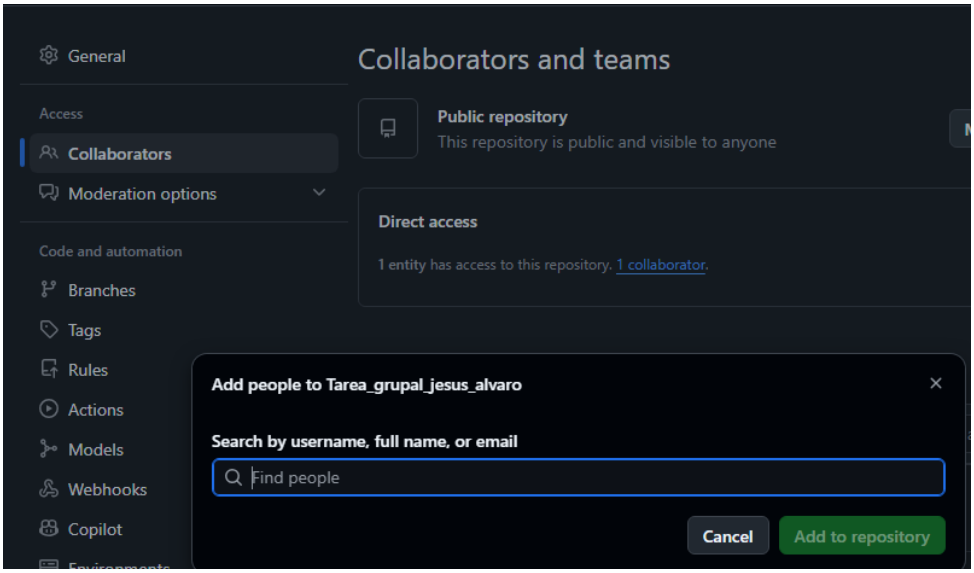
Para gestionar el ciclo de vida del desarrollo y asegurar la calidad del código, se ha configurado un repositorio en GitHub siguiendo las convenciones de Git. La estructura de ramas se ha diseñado de la siguiente manera:

Master:

Contiene únicamente el código original inalterado para permitir la comparación del estado inicial y final.

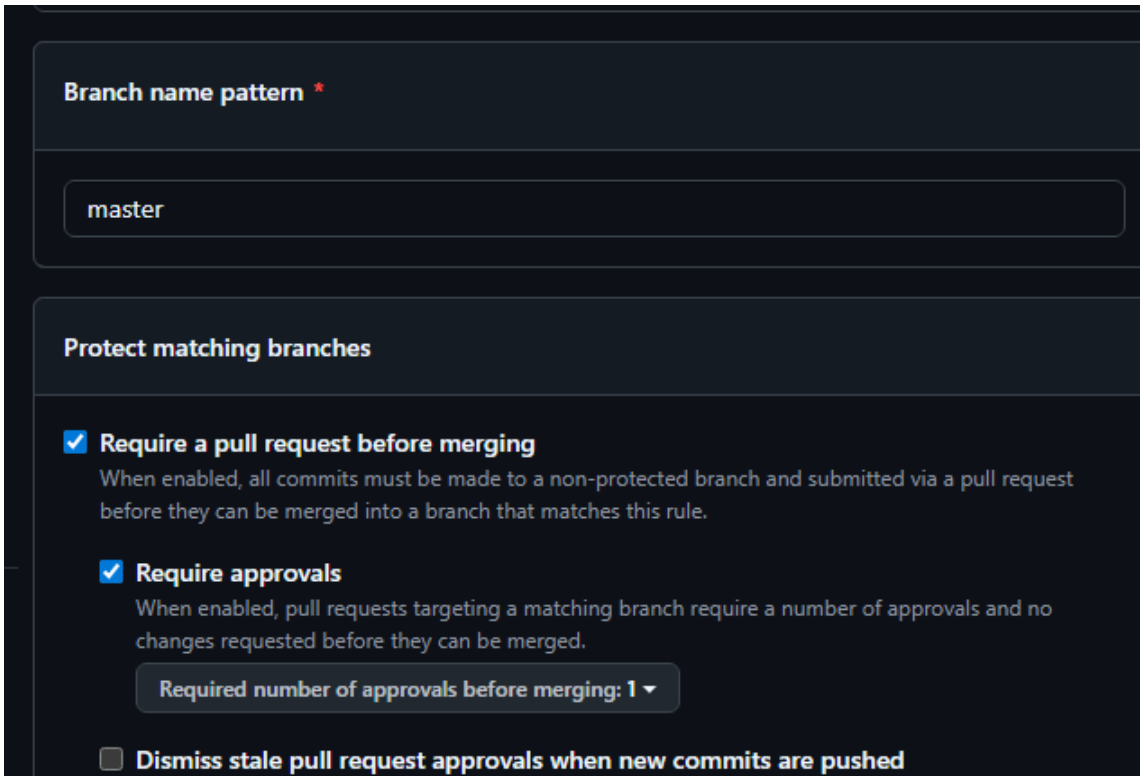
Añadir al compañero:

En este paso buscaríamos a nuestro compañero en “Find people” y le daríamos a “Add to repository”



Protección de Ramas:

Para garantizar el control de calidad, se ha protegido la rama master obligando al uso de Pull Requests para la integración de código. Además, se ha configurado el equipo de trabajo añadiendo a los miembros como colaboradores.



develop:

Rama de integración (`develop`) donde se fusionan las refactorizaciones funcionales.

Creación de la rama de integración: Se inicializó la rama develop a partir de master (git checkout -b develop)

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git checkout -b develop
Switched to a new branch 'develop'
```

se vinculó con el repositorio remoto (git push -u origin develop). Esta rama actúa como punto de encuentro para el trabajo de todos los miembros del equipo.

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro/pull/new/develop
remote:
To https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

feature:

Ramas de trabajo aisladas para cada patrón, como feature/alvaro-extraer-metodos y feature/alvaro-polimorfismo.

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git checkout -b feature/refactor-notificaciones
Switched to a new branch 'feature/refactor-notificaciones'
```

Commits atómicos: Una vez preparado el código base en el área de trabajo, se utilizó git add . para empaquetar los cambios y se realizó un git commit.

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git add .
```

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git commit -m "feat: añadir codigo base original del project04"
[feature/refactor-notificaciones 1bacda0] feat: añadir codigo base original del project04
 1 file changed, 14 insertions(+)
 create mode 100644 src/com/refactoring/projects/project04/NotificationService.java.txt
```

Sincronización remota: Finalmente, se subió la rama de trabajo al repositorio remoto en GitHub mediante el comando git push, dejándola lista para su posterior revisión y fusión (Pull Request) hacia develop.

```
C:\Users\abarb\Tarea_grupal_jesus_alvaro>git push origin feature/refactor-notificaciones
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (8/8), 799 bytes | 799.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/refactor-notificaciones' on GitHub by visiting:
remote:      https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro/pull/new/feature/refactor-notificaciones
remote:
To https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro.git
 * [new branch]      feature/refactor-notificaciones -> feature/refactor-notificaciones
```

2. Identificación de Patrones para Clean Code (Tarea 2)

Tras analizar el código fuente original de `NotificationService`, se identificaron varios "Code Smells" principalmente Hardcodeo y la excesiva complejidad ciclomática mediante bloques condicionales extensos.

Para lograr un *Clean Code* y cumplir con el requisito de aplicar al menos 3 patrones, se justificó la aplicación de los siguientes patrones de refactorización:

1. **Extraer constantes**: En el código original, los tipos de notificación ("email", "sms", "push") estaban escritos a fuego. Se aplicó el patrón de extraer constantes para eliminar datos hardcodeados, evitando errores tipográficos y facilitando la modificación futura del código.

2. **Extraer métodos:** La lógica de imprimir por pantalla los mensajes estaba junta en el método principal. Según la teoría, es necesario extraer métodos cuando un bloque de código asume demasiadas responsabilidades. Debido a esto se extrajo la lógica a métodos privados individuales (enviarEmail, enviarSMS, enviarPush) para hacer el código más mantenible.

3. **Reemplazar condicionales por polimorfismo:** El método principal contenía una estructura condicional que no permitia muchos cambios, ya que la clase necesitaría ser modificada a menudo para añadir nuevos tipos de mensaje.

   Para solucionarlo, se aplicó el patrón de reemplazar implementaciones condicionales por polimorfismo Para lograr esto, primero hubo que extraer una interfaz común, delegando así el comportamiento de envío a las clases hijas.

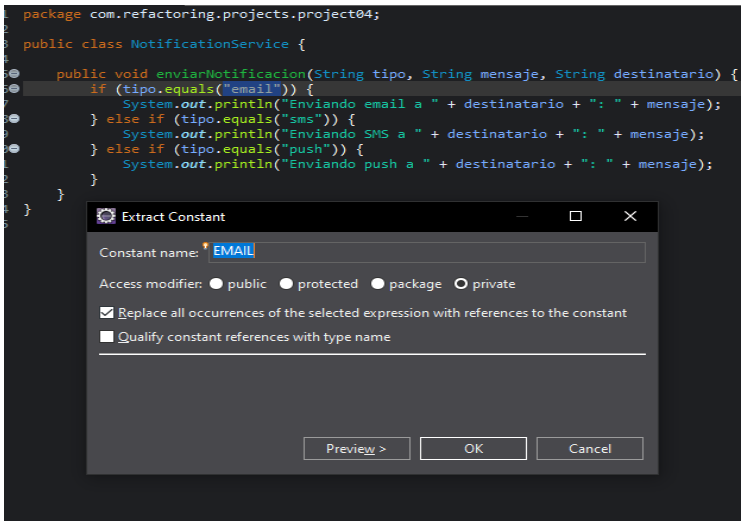
3. Implementación de la Refactorización (Tarea 3)

3.1. Extracción de Constantes

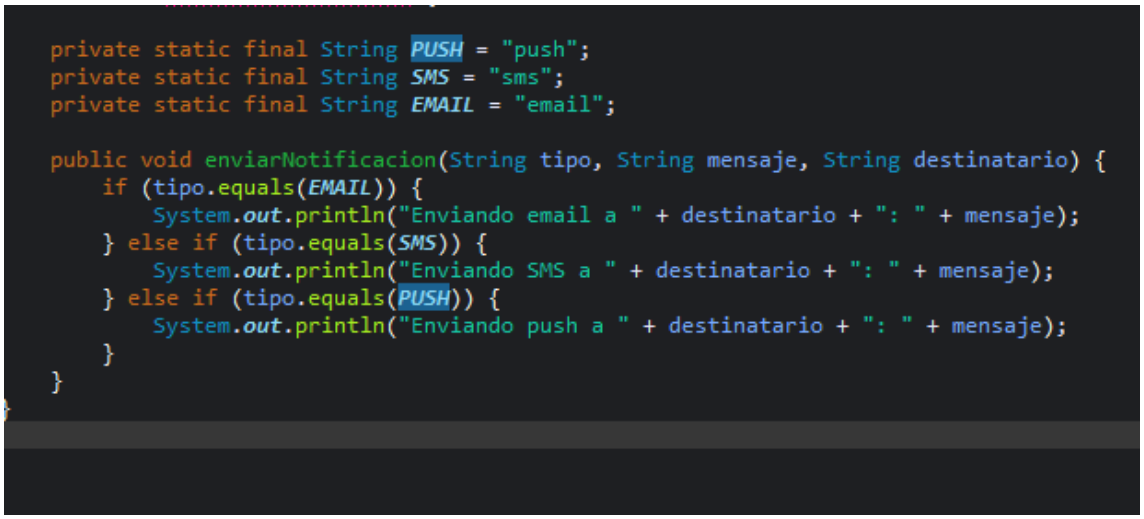
Antes de extraer métodos.

```java
package com.refactoring.projects.project04;

public class NotificationService {

    public void enviarNotificacion(String tipo, String mensaje, String destinatario)
        if (tipo.equals("email")) {
            System.out.println("Enviando email a " + destinatario + ": " + mensaje);
        } else if (tipo.equals("sms")) {
            System.out.println("Enviando SMS a " + destinatario + ": " + mensaje);
        } else if (tipo.equals("push")) {
            System.out.println("Enviando push a " + destinatario + ": " + mensaje);
        }
    }
```

Clickamos el variable que queramos cambiar. Le damos a: click derecho > Refactor > Extract Constant, como se ve en la imagen de abajo



Hacemos este proceso en “sms” y push, una vez hayamos acabado nos debe de quedar un resultado tal que asi:



```
    private static final String PUSH = "push";
    private static final String SMS = "sms";
    private static final String EMAIL = "email";

    public void enviarNotificacion(String tipo, String mensaje, String destinatario) {
        if (tipo.equals(EMAIL)) {
            System.out.println("Enviando email a " + destinatario + ": " + mensaje);
        } else if (tipo.equals(SMS)) {
            System.out.println("Enviando SMS a " + destinatario + ": " + mensaje);
        } else if (tipo.equals(PUSH)) {
            System.out.println("Enviando push a " + destinatario + ": " + mensaje);
        }
    }
}
```

Y aplicamos los cambios recién aplicados a través de git:

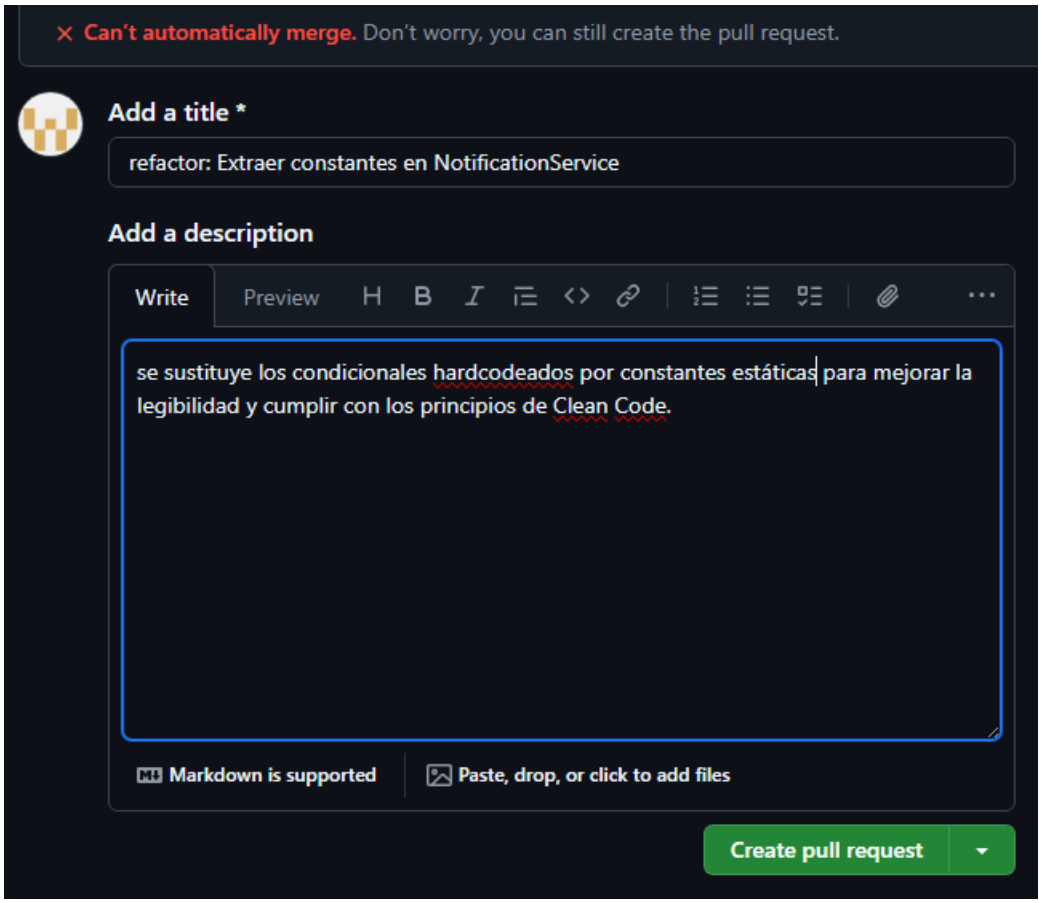
Creamos los commits

```
MINGW64:/c/Users/j32u2/Tarea_grupal_jesus_alvaro

j32u2@YisusCompu MINGW64 ~
$ cd Tarea_grupal_jesus_alvaro/

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git checkout develop_jesus
M       src/com/refactoring/projects/project04/NotificationService.java
Already on 'develop_jesus'
Your branch is up to date with 'origin/develop_jesus'.

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git pull origin develop_jesus
From https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro
 * branch            develop_jesus -> FETCH_HEAD
Already up to date.

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git add .

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git commit -m "Refactorizacion Paso 1.2: Extraer constante llamada SMS para el
iminar el dato Hardcodeado"
[develop_jesus 110b247] Refactorizacion Paso 1.2: Extraer constante llamada SMS
para eliminar el dato Hardcodeado
 1 file changed, 4 insertions(+), 4 deletions(-)

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git add .

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git commit -m "Refactorizacion Paso 1.3: Extraer constante llamada Push para e
liminar el dato Hardcodeado"
[develop_jesus c9a6ae5] Refactorizacion Paso 1.3: Extraer constante llamada Push
 para eliminar el dato Hardcodeado
 1 file changed, 3 insertions(+), 2 deletions(-)

j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ |
```

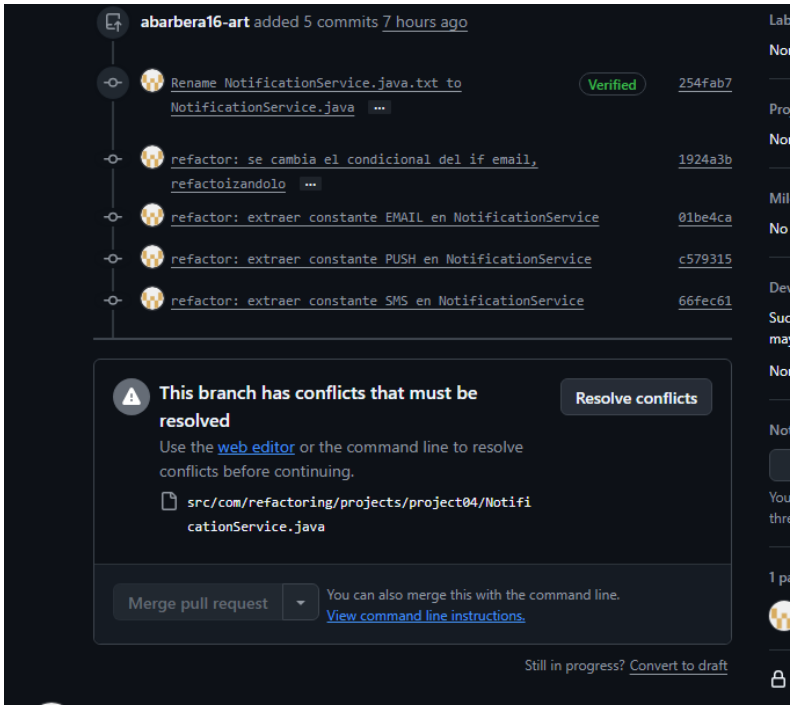
Enviamos los commits al directorio de github

```
j32u2@YisusCompu MINGW64 ~/Tarea_grupal_jesus_alvaro (develop_jesus)
$ git push
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 24 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (16/16), 1.16 KiB | 1.16 MiB/s, done.
Total 16 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To https://github.com/abarbera16-art/Tarea_grupal_jesus_alvaro.git
   a580026..c9a6ae5  develop_jesus -> develop_jesus
```

Vamos a nuestro repositorio de github y vemos q no es posible un mergeo automatico, debido a diferencias en el codigo.



Pero Github nos ofrece una de sus funcionalidades que mas usaremos en este trabajo que es: la resolución de conflictos entre el código



Clickamos en resolve conflicts y vemos como github nos presenta el código actual y el cambio que acabamos de proponer, seleccionamos el código nuevo

src/com/refactoring/projects/project04/NotificationService.java

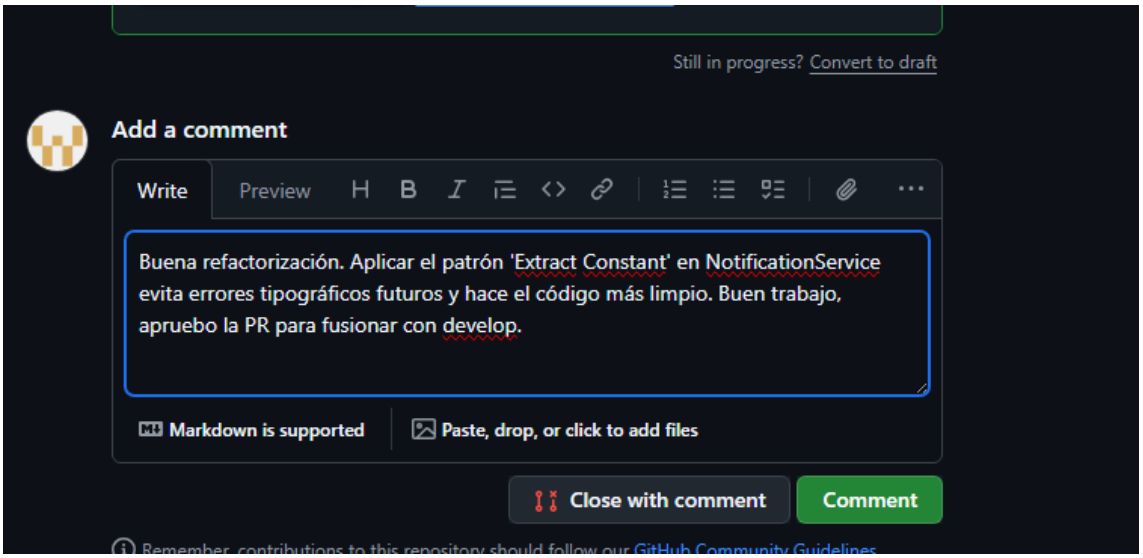
1 conflict Prev Next Mark as resolved

```
1   package com.refactoring.projects.project04;
2
3   public class NotificationService {
4
    Accept current change | Accept incoming change | Accept both changes
5   <<<<<<< feature/alvaro-refactor (Current change)
6     private static final String SMS = "sms";
7     private static final String PUSH = "push";
8     private static final String EMAIL = "email";
9
10    public void enviarNotificacion(String tipo, String mensaje, String destinatario) {
11          if (tipo.equals(EMAIL)) {
12              System.out.println("Enviando email a " + destinatario + ": " + mensaje);
13          } else if (tipo.equals(SMS)) {
14              System.out.println("Enviando SMS a " + destinatario + ": " + mensaje);
15          } else if (tipo.equals(PUSH)) {
16              System.out.println("Enviando push a " + destinatario + ": " + mensaje);
17          }
18      }
19
20  =======
21    public void enviarNotificacion(String tipo, String mensaje, String destinatario) {
22          if (tipo.equals("email")) {
23              System.out.println("Enviando email a " + destinatario + ": " + mensaje);
24          } else if (tipo.equals("sms")) {
25              System.out.println("Enviando SMS a " + destinatario + ": " + mensaje);
26          } else if (tipo.equals("push")) {
27              System.out.println("Enviando push a " + destinatario + ": " + mensaje);
28          }
29      }
30  >>>>>>> develop_alvaro (Incoming change)
31  }
32
```

(captura de ejemplo)

Después de solucionar el conflicto clickamos en Commit merge y ya habríamos mergeado el cambio con el código de develop

Añadimos un comentario simulando una revisión del codigo igual que lo haríamos a la hora de mergear con el master



(Todo este proceso se haría con todos los pasos)

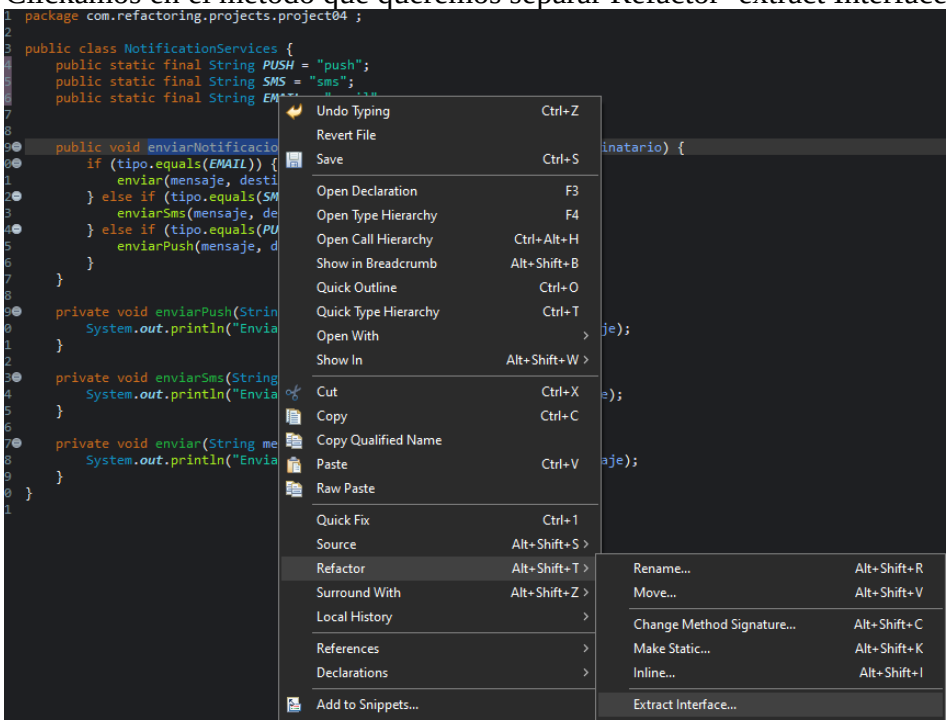
3.2. Extracción de Métodos

Antes de extraer métodos.Seleccionamos la línea de código `System.out.println` que queramos cambiar. Le damos a: click derecho > Refactor > Extract Method y le ponemos el nombre enviarEmail en este caso , como se ve en la imagen de abajo

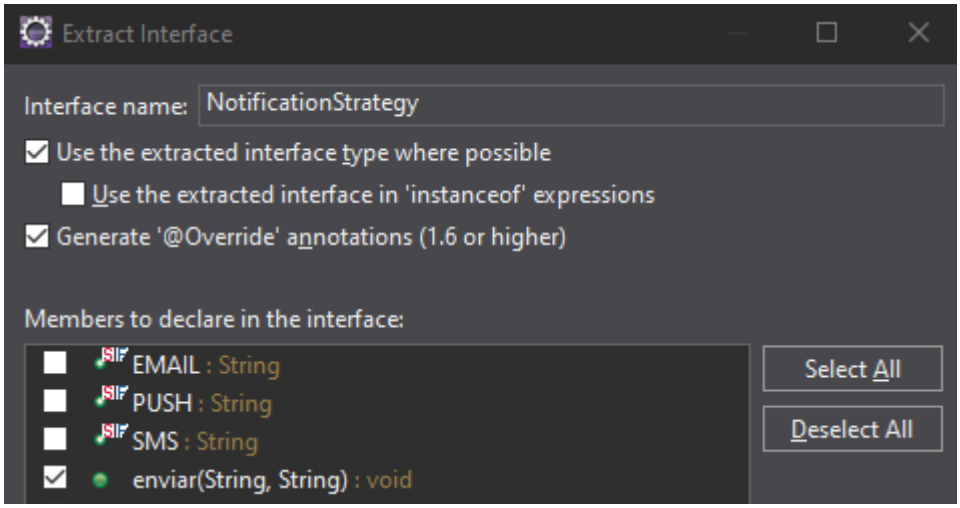
*NotificationServices.java NotificacionStrategy.java

```java
package com.refactoring.projects.project04;

public class NotificationServices {
    public static final String PUSH = "push";
    public static final String SMS = "sms";
    public static final String EMAIL = "email";


    public void enviarNotificacion(String tipo, String mensaje, String destinatario) {
        if (tipo.equals(EMAIL)) {
            enviar(mensaje, destinatario);
        } else if (tipo.equals(SMS)) {
            enviarSms(mensaje, destinatario);
        } else if (tipo.equals(PUSH)) {
            enviarPush(mensaje, destinatario);
        }
    }

    private void enviarPush(String mensaje, String destinatario) {
        System.out.println("Enviando push a " + destinatario + ": " + mensaje);
    }

    private void enviarSms(String mensaje, String destinatario) {
        System.out.println("Enviando SMS a " + destinatario + ": " + mensaje);
    }


    public void enviar(String mensaje, String destinatario) {
        System.out.println("Enviando email a " + destinatario + ": " + mensaje);
    }
}
```

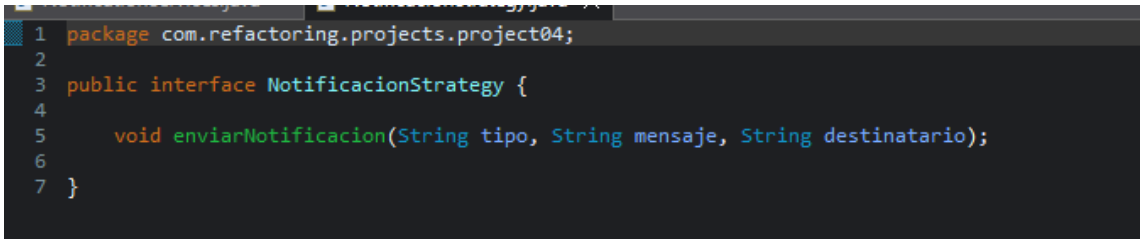
Clickamos en el método que queremos separar Refactor>extract Interface



Seleccionamos el nombre de la interfaz y atributos que queremos que tenga la interfaz



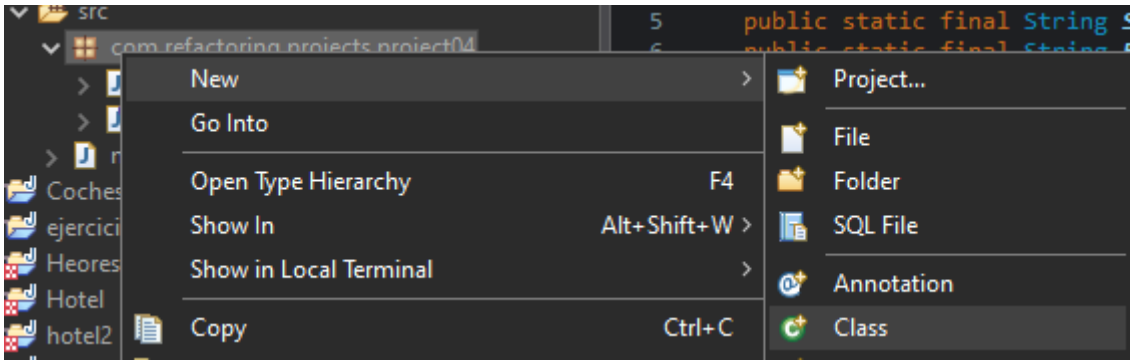
Este es nuestro resultado, la interfaz lista para su uso



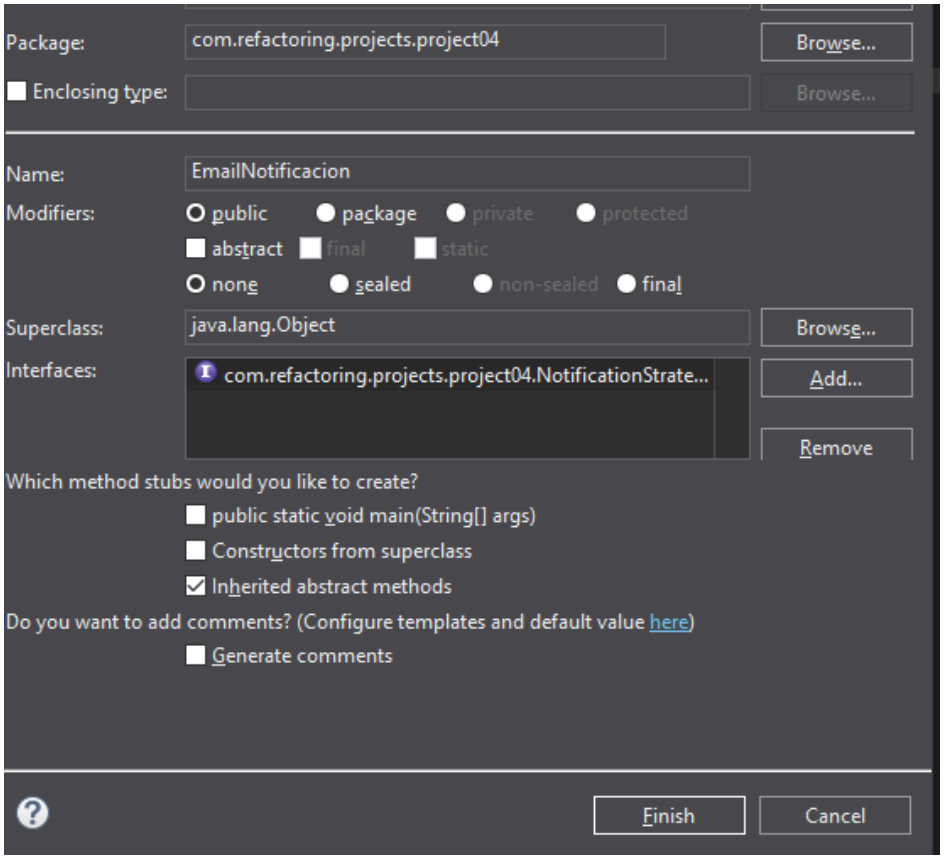
```
1 package com.refactoring.projects.project04;
2
3 public interface NotificacionStrategy {
4
5     void enviarNotificacion(String tipo, String mensaje, String destinatario);
6
7 }
```

Ahora para crear las clases en las que se apoyara la interfaz seguiremos estos pasos

Clickamos en el paquete new>class



Seleccionamos el nombre y la interfaz que va a usar



Hacemos este proceso en la lógica de "sms" y "push" (nombrándolos enviarSMS y enviarPush), una vez hayamos acabado nos debe de quedar un resultado tal que asi en todas las clases:

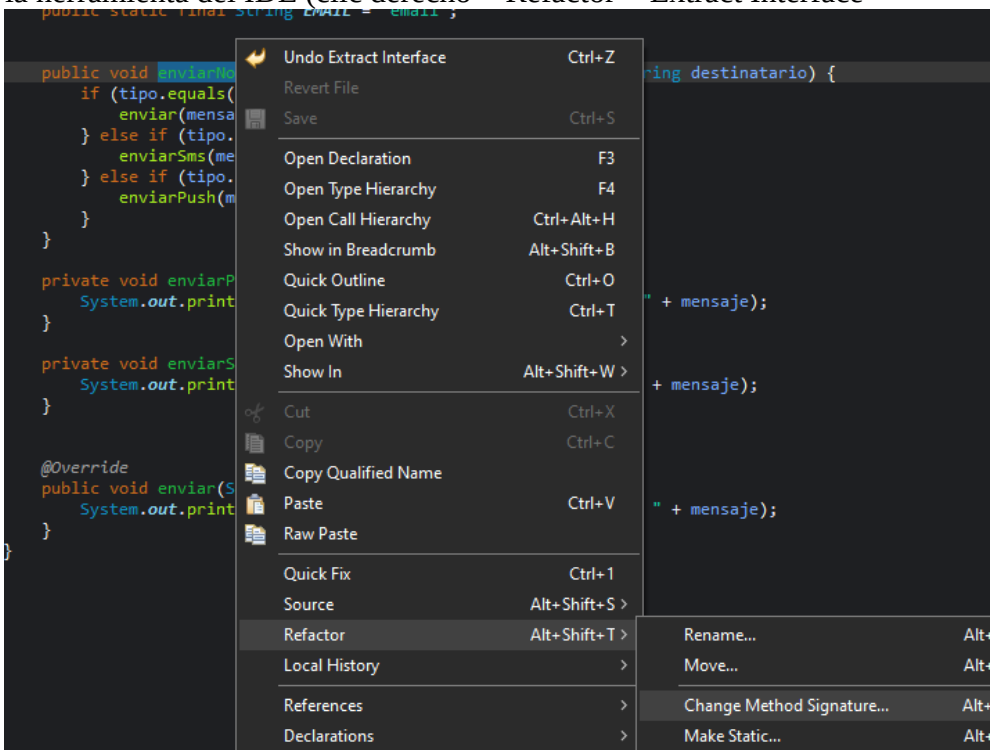
```java
package com.refactoring.projects.project04;

public class EmailNotificacion implements NotificationStrategy {

    @Override
    public void enviar(String mensaje, String destinatario) {
        // TODO Auto-generated method stub

    }

}
```

```java
package com.refactoring.projects.project04;

public class SmsNotification implements NotificationStrategy {

    @Override
    public void enviar(String mensaje, String destinatario) {
        // TODO Auto-generated method stub

    }

}
```

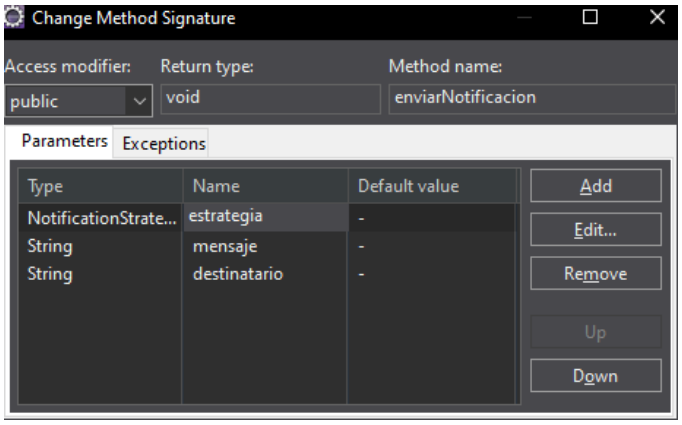
```java
package com.refactoring.projects.project04;

public class PushNotification implements NotificationStrategy {

    @Override
    public void enviar(String mensaje, String destinatario) {
        // TODO Auto-generated method stub

    }

}
```

3.3. Reemplazar condicionales por polimorfismo

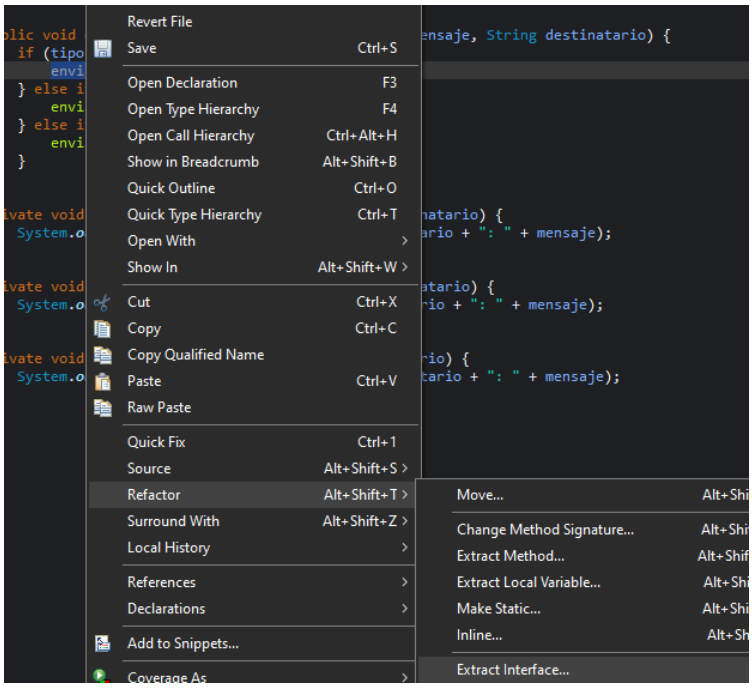
Para eliminar los bloques if-else, extrajimos la interfaz NotificacionStrategy utilizando la herramienta del IDE (clic derecho > Refactor > Extract Interface



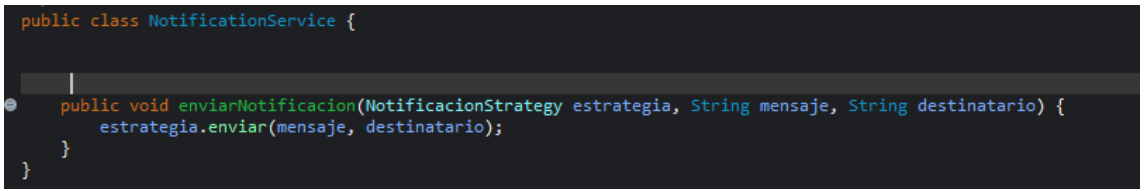
Le damos su nuevo nombre al método.



Refactorización con Polimorfismo. Se utiliza la herramienta del IDE (Refactor > Extract Interface) para generar automáticamente la interfaz NotificationStrategy.



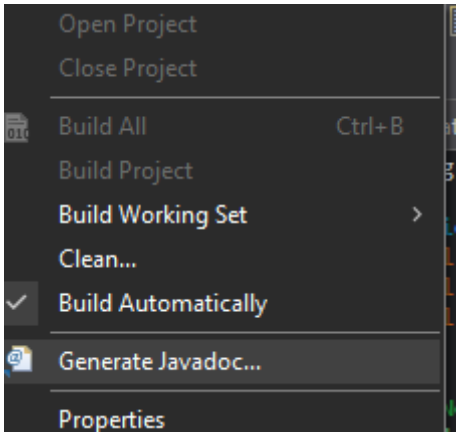
Se modifica la clase principal para eliminar los condicionales if-else, pasando la nueva interfaz por parámetro y delegando la acción estrategia.enviar para cumplir con los principios Clean Code.



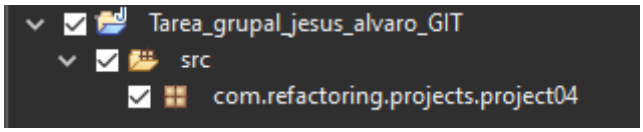
```
public class NotificationService {

    public void enviarNotificacion(NotificacionStrategy estrategia, String mensaje, String destinatario) {
        estrategia.enviar(mensaje, destinatario);
    }
}
```

Generar Javadoc

Como ultimo paso generaremos el Javadoc para extraer todos los comentarios técnicos que hemos ido añadiendo en el código (etiquetas como @author, @version y @param) y convertirlos en una página web navegable



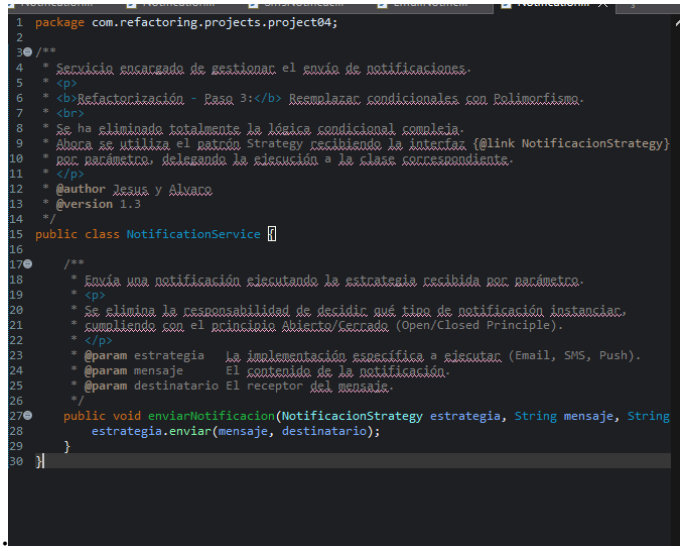
Seleccionamos el proyecto del cual queremos crear el Java doc y listo!



Documentación avanzada con el IDE y formato HTML

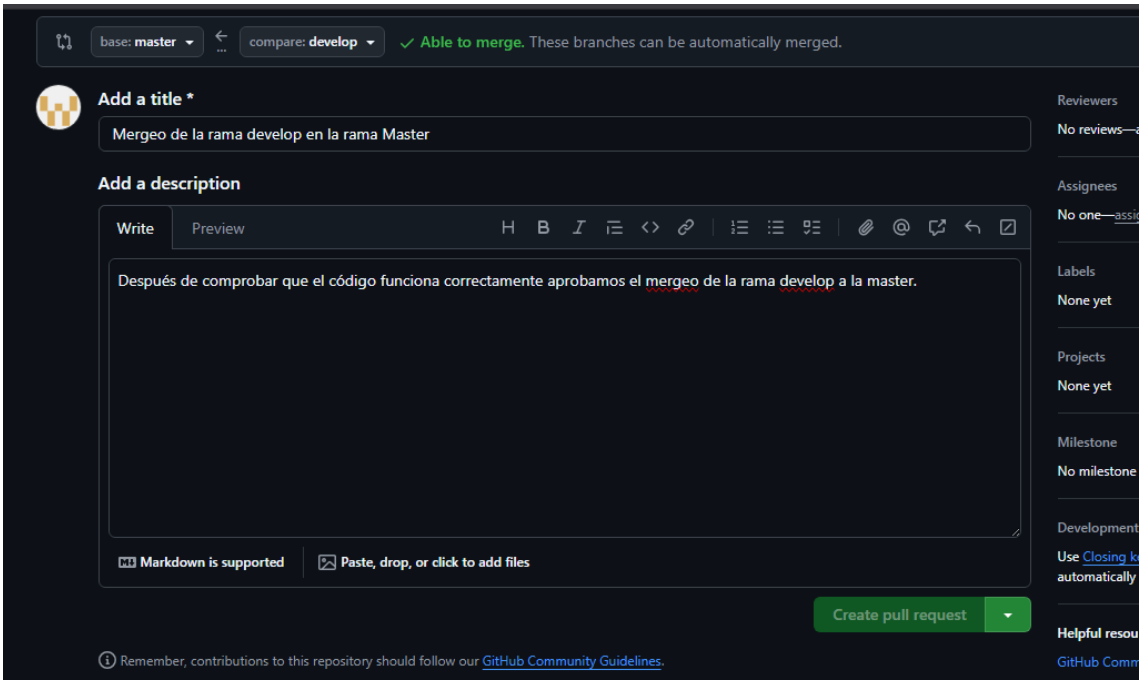
La siguiente captura ilustra el proceso de documentación de la clase principal NotificationService aprovechando las facilidades del IDE. Al abrir un bloque de JavaDoc (/** y pulsar Enter), el entorno nos ha generado automáticamente las etiquetas @param correspondientes a la firma del método

Además, para enriquecer la presentación del documento web resultante, hemos integrado manualmente etiquetas de HTML estándar dentro de los propios comentarios, asegurando así. Una lectura profesional

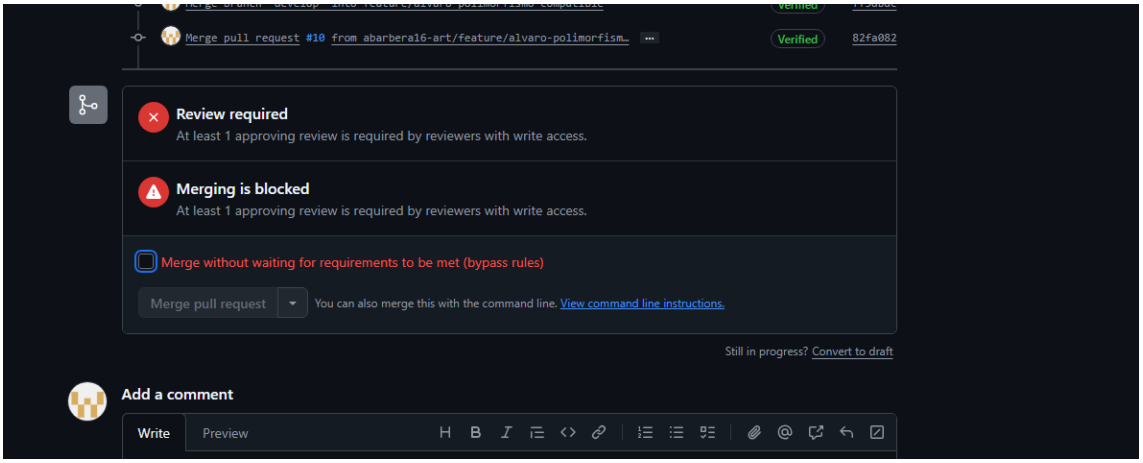


```
1  package com.refactoring.projects.project04;
2
3  /**
4   * Servicio encargado de gestionar el envío de notificaciones.
5   * <p>
6   * <b>Refactorización - Paso 3:</b> Reemplazar condicionales con Polimorfismo.
7   * <br>
8   * Se ha eliminado totalmente la lógica condicional compleja.
9   * Ahora se utiliza el patrón Strategy recibiendo la interfaz {@link NotificacionStrategy}
10  * por parámetro, delegando la ejecución a la clase correspondiente.
11  * </p>
12  * @author Jesus y Alvaro
13  * @version 1.3
14  */
15 public class NotificationService {
16
17     /**
18      * Envía una notificación ejecutando la estrategia recibida por parámetro.
19      * <p>
20      * Se elimina la responsabilidad de decidir qué tipo de notificación instanciar,
21      * cumpliendo con el principio Abierto/Cerrado (Open/Closed Principle).
22      * </p>
23      * @param estrategia   La implementación específica a ejecutar (Email, SMS, Push).
24      * @param mensaje      El contenido de la notificación.
25      * @param destinatario El receptor del mensaje.
26      */
27     public void enviarNotificacion(NotificacionStrategy estrategia, String mensaje, String
28         estrategia.enviar(mensaje, destinatario);
29     }
30 }
```

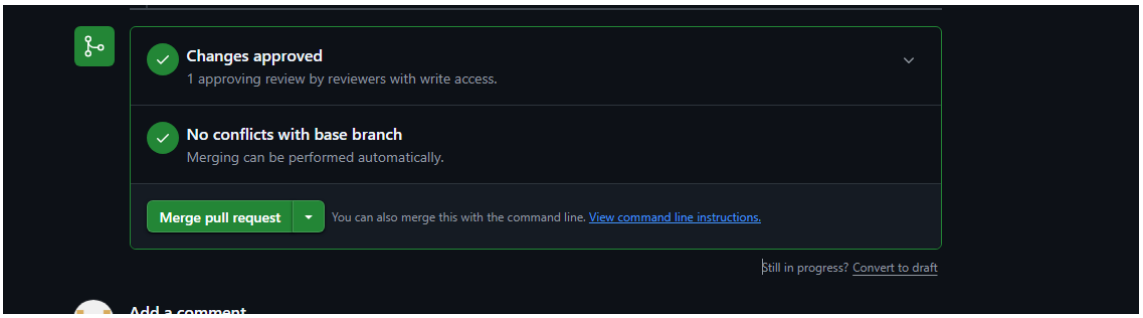
Despues de hacer todo estos procedimientos con todo el código llevamos acabo el mergeo de la rama develop en la rama master



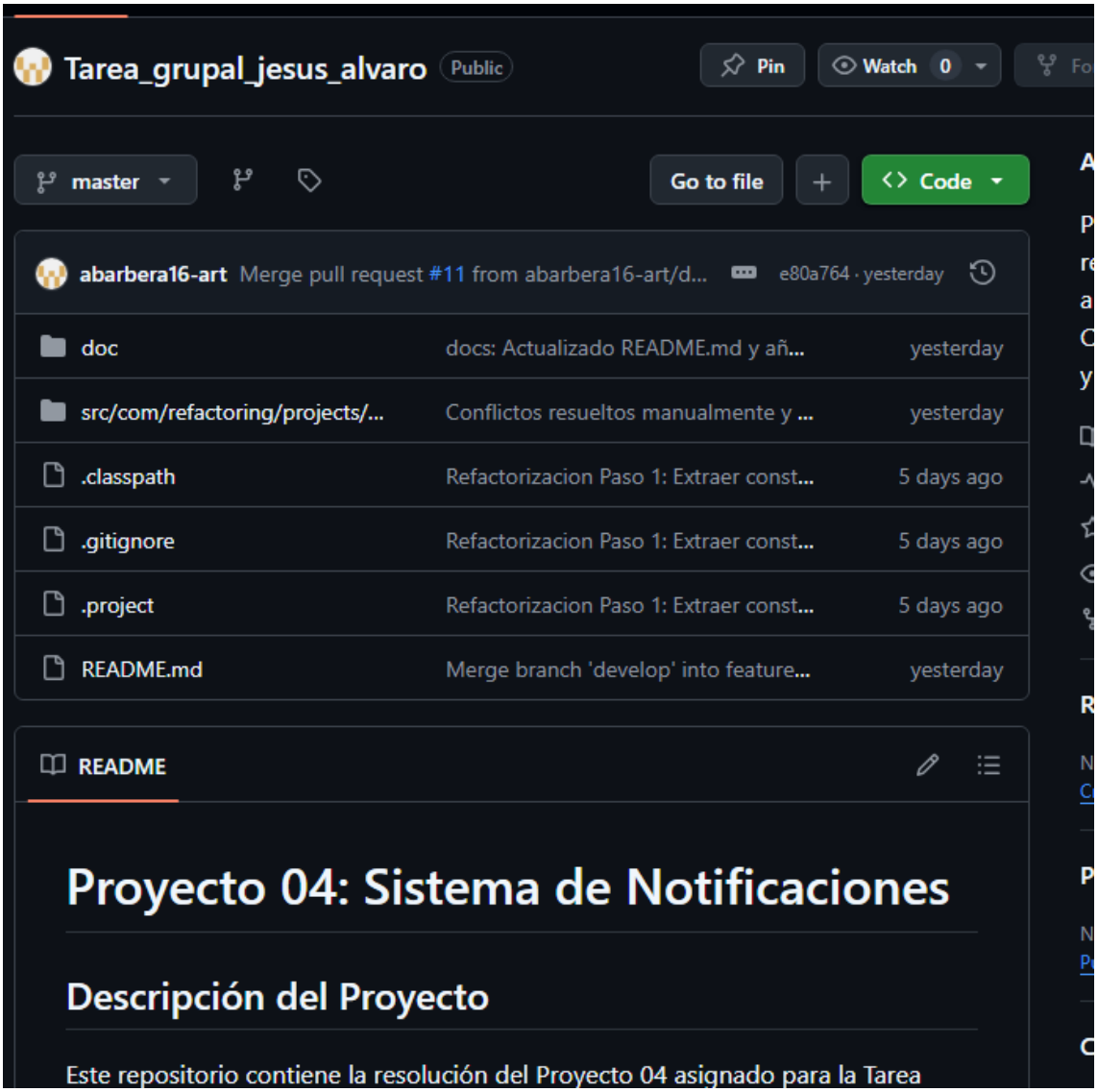
Aquí nuestro compañero tendría que aprobar el mergeo debido a a las reglas de protección de rama que configuramos al inicio del proyecto, las cuales bloquean la fusión automática y exigen al menos una revisión aprobatoria



Una vez nuestro compañero escriba la review podremos proceder al mergeo



Integración Final y Conclusión (Merge a Master)

Como culminación de nuestro flujo de trabajo la siguiente captura muestra el estado definitivo del repositorio. Una vez validadas todas las refactorizaciones y resueltos los conflictos en la rama de integración, se procedió a realizar el Pull Request final hacia la rama master

En esta versión estable ya se encuentra integrado todo el Clean Code, la carpeta doc resultante de la generación del JavaDoc, y un archivo README.md actualizado con la descripción completa del proyecto.